

MAF/MADE Residual and Masking Notes (TTZ)

March 30, 2026

1 Scope of This Note

This note is only about the architecture discussion: how masking works in MADE, how residual/skip layers are implemented in the TTZ flow model, and how these pieces combine inside MAF/MAFMADEMOG.

The active TTZ settings discussed were:

- `num_flows = 5`
- `num_hidden_layers = 4`
- `hidden_layer_dim = 384`
- `res_layers_in_block = 2`
- `conv1x1 = True`
- `maf_residuals = True`

2 Big Picture: What Problem MAF Solves

Normalizing flows learn an invertible map between data x and latent u with tractable log-density:

$$\log p_X(x) = \log p_U(u) + \log \left| \det \frac{\partial u}{\partial x} \right|.$$

In MAF, each block is autoregressive, so the Jacobian is triangular and its log-determinant is cheap to compute.

The model gains flexibility by stacking multiple autoregressive transforms, interleaved with invertible mixing/permutation layers.

3 How Autoregressive Masking Works in MADE

Core rule

Each output dimension i is only allowed to depend on previous dimensions in a chosen order. In this codebase the ordering is sequential by default.

The masking function constructs binary masks so that illegal connections are exactly zeroed out in `MaskedLinear`:

$$\text{out} = W \odot M x + b.$$

So masking is not a soft regularizer. It is a hard structural constraint.

Degree-based construction

The implementation assigns each neuron a degree m and allows connections only from earlier/equal degrees according to the MADE rules:

- hidden-layer masks use a *non-strict* condition ($\geq$),
- output mask uses a *strict* condition ($>$).

That strict final mask is what guarantees output i cannot use its own input x_i directly in the autoregressive parameter prediction.

4 Gaussian Parameterization Used in MAF Blocks

The Gaussian MADE variants output two vectors per feature:

$$\mu(x_{<i}), \quad \alpha(x_{<i})$$

where α is log-scale. The transform used in normalizing direction is:

$$u_i = (x_i - \mu_i) \exp(-\alpha_i).$$

Then:

$$\log \left| \det \frac{\partial u}{\partial x} \right| = - \sum_i \alpha_i.$$

Sampling (inverse direction) is sequential in feature index because each x_i needs already-generated previous coordinates.

5 Residual Blocks: What “Skipping” Means Here

Form of the residual update

Inside `ResMaskedLinear`, the block computes:

$$y = x + f(x),$$

where f is a masked stack of nonlinear layers.

This is the skip connection we discussed. The identity path (x) is always present, and the block learns a correction $f(x)$ instead of a full remapping.

Why this helps

Residual form improves optimization because:

- gradients can flow through the identity branch,
- deep stacks are less prone to representational collapse,
- near-identity behavior is easy early in training.

Important implementation detail: zero initialization

The final layer inside each residual mini-network is zero-initialized in `ResMaskedLinear` (weights and bias).

Consequence at initialization:

$$f(x) \approx 0 \quad \Rightarrow \quad y \approx x.$$

So each residual block starts near identity and gradually learns structured deviations. This typically stabilizes early training dynamics.

6 Two Depth Knobs You Asked About

In `GaussianResMADE(input_size, k, n_blocks, l)`:

- `n_blocks` is the number of residual blocks.
- `l` is the number of masked layers inside each residual block.

Mapping to TTZ config in this repository:

- `n_blocks = num_hidden_layers = 4`
- `l = res_layers_in_block = 2`
- `k = hidden_layer_dim = 384`

So one autoregressive block contains 4 residual units, each with 2 internal masked layers, all at width 384.

7 How One Full MAFMADEMOG Block Is Built

For each of the `num_flows` repeats, the code adds:

1. optional `BatchNormFlow`,
2. either `Conv1x1PLU` (if `conv1x1=True`) or `ReverseFlow`,
3. an autoregressive transform: `GaussianResMADE` if residuals are enabled.

With your settings, this means each repeat uses batch norm, then invertible 1x1 mixing, then residual autoregression.

8 Why 1x1 Invertible Mixing Is Useful with Autoregression

Autoregressive layers are order-sensitive. Without mixing/permutation, later blocks repeatedly see very similar coordinate ordering constraints.

The invertible 1x1 linear transform rotates/mixes channels between blocks, which changes what each next autoregressive layer sees as “earlier” vs “later” information content.

Intuition: masking gives triangular structure; 1x1 mixing prevents that structure from becoming too rigid across depth.

9 Forward vs Inverse Cost Asymmetry (Important Practical Point)

In autoregressive flows:

- density evaluation in normalizing direction is parallel over dimensions,
- sampling in inverse direction is sequential over dimensions.

This is why MAF is typically efficient for likelihood training and comparatively slower for generation than coupling-based alternatives.

10 How Skip Connections and Masks Interact

A common confusion is whether residual skips break autoregressive validity. They do not, because:

- the residual branch itself uses masked linear layers,
- the identity add $x + f(x)$ is within hidden feature space of fixed width,
- the output projection still uses the autoregressive output mask.

So residual design improves optimization capacity while preserving the same autoregressive dependency constraints.

11 Concrete Parameter Counting Intuition (Qualitative)

Increasing these knobs does the following:

- `hidden_layer_dim` increases per-layer capacity strongly.
- `num_hidden_layers` increases number of residual units per MAF block.
- `res_layers_in_block` deepens each residual unit.
- `num_flows` repeats the whole (mixing + autoregressive) macro-block.

In practice, these knobs trade off expressivity, train time, memory use, and stability. Residual and near-identity initialization are key enablers when depth is increased.

12 Practical Reading of Your Current Setup

Given your TTZ settings (5 flow repeats, residual MADE blocks, width 384, 4 residual blocks each with 2 internal masked layers, and invertible 1x1 mixing):

- the model has substantial capacity for nontrivial correlations,
- residualization should help keep training stable at this depth,
- 1x1 mixing should reduce repeated-order artifacts across flow repeats.

This is exactly the architecture pattern one would choose when simple autoregressive stacks are underfitting correlation structure.

13 Worked Example: One Event Through One Flow Repeat

This section gives a concrete step-by-step pass for a single event vector $x \in \mathbb{R}^{15}$ (TTZ input dimension in the current setup).

Assume we are in one repeat of the flow stack with:

1. BatchNormFlow,
2. Conv1x1PLU,
3. GaussianResMADE.

Step 1: Batch normalization transform

Input event is $x^{(0)} = x$. Batch norm applies a per-feature affine normalization:

$$x_j^{(1)} = w_j \frac{x_j^{(0)} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} + b_j.$$

This contributes an additive log-Jacobian term $\log |\det J_{\text{BN}}|$ that is easy to compute from per-feature scales.

Interpretation: this step re-centers/re-scales channels so later layers operate in a numerically friendlier regime.

Step 2: Invertible 1x1 linear mixing

Then Conv1x1PLU applies an invertible linear map:

$$x^{(2)} = x^{(1)} W,$$

where W is parameterized through PLU factors for stable invertibility and cheap log-determinant:

$$\log |\det J_{\text{mix}}| = \log |\det W|.$$

Interpretation: coordinates are rotated/mixed so the next autoregressive module does not always see the same axis-aligned dependency pattern.

Step 3: Residual autoregressive map (GaussianResMADE)

Now GaussianResMADE receives $x^{(2)}$ and predicts two 15-vectors:

$$\mu(x_{<i}^{(2)}), \quad \alpha(x_{<i}^{(2)}).$$

Each hidden computation inside this network uses masked linear layers. Residual blocks apply hidden-state updates of the form:

$$h \leftarrow h + f(h),$$

with masked f . Because the final masked layer in each residual unit is zero-initialized, each unit starts near identity at initialization.

The autoregressive transform for each feature is:

$$u_i = (x_i^{(2)} - \mu_i) \exp(-\alpha_i).$$

Collecting all features gives $u^{(3)}$ and log-Jacobian term:

$$\log |\det J_{\text{AR}}| = - \sum_i \alpha_i.$$

Step 4: Total contribution of this repeat

For this one repeat, output is $u^{(3)}$, and the repeat contributes:

$$\log |\det J_{\text{repeat}}| = \log |\det J_{\text{BN}}| + \log |\det J_{\text{mix}}| + \log |\det J_{\text{AR}}|.$$

This output is then fed to the next repeat (out of 5 total in your config).

What changes during sampling (inverse direction)

During generation, the inverse order is used inside each repeat:

1. inverse autoregressive step (sequential in feature index),
2. inverse 1x1 mixing,
3. inverse batch norm.

The sequential part is specifically the autoregressive inverse:

$$x_i = u_i \exp(\alpha_i) + \mu_i,$$

where μ_i, α_i depend on already reconstructed earlier coordinates.

This is the core reason sampling is slower than density evaluation in MAF.

14 Minimal Mental Model

If you want one compact memory aid:

MADE masks decide *who can talk to whom*. Residual skips decide *how hard optimization has to work*. 1x1 invertible mixing decides *whether each new autoregressive block sees a fresh coordinate geometry*.

Together, they give an expressive but still tractable flow for high-dimensional physics features.